Revision of OOP in Java

Discipline: Computer Science III

Scope

- Object Oriented Programming in Java
- Classes and Objects
- Inheritance
- Polymorphism

Object Oriented Programming in Java

Java is an object-oriented programming language

- As the term implies, an object is a fundamental entity in a Java program
- Objects can be used effectively to represent real-world entities

Objects

An object has

- state – descriptive characteristics - attributes
- *behaviors* - what it can do (or what can be done to it) - methods

The state of a bank account includes its account number and its current balance

The behaviors associated with a bank account include the ability to make deposits and withdrawals

- Note that the behavior of an object often changes its state

Classes

An object is defined by a *class*

A *class* is the blueprint of an object

The class uses *methods* to define the behaviors of the object

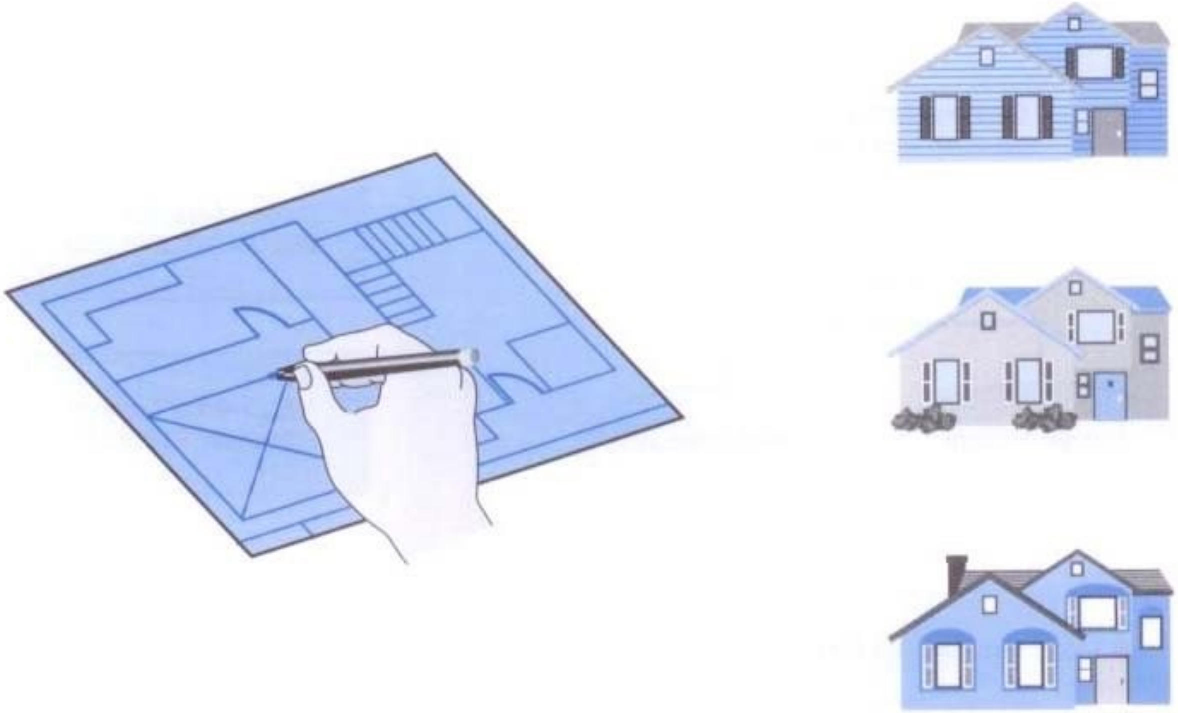
The class that contains the ***`main`*** method of a Java program represents the entire program

A class represents a concept, and an object represents an instantiation of that concept

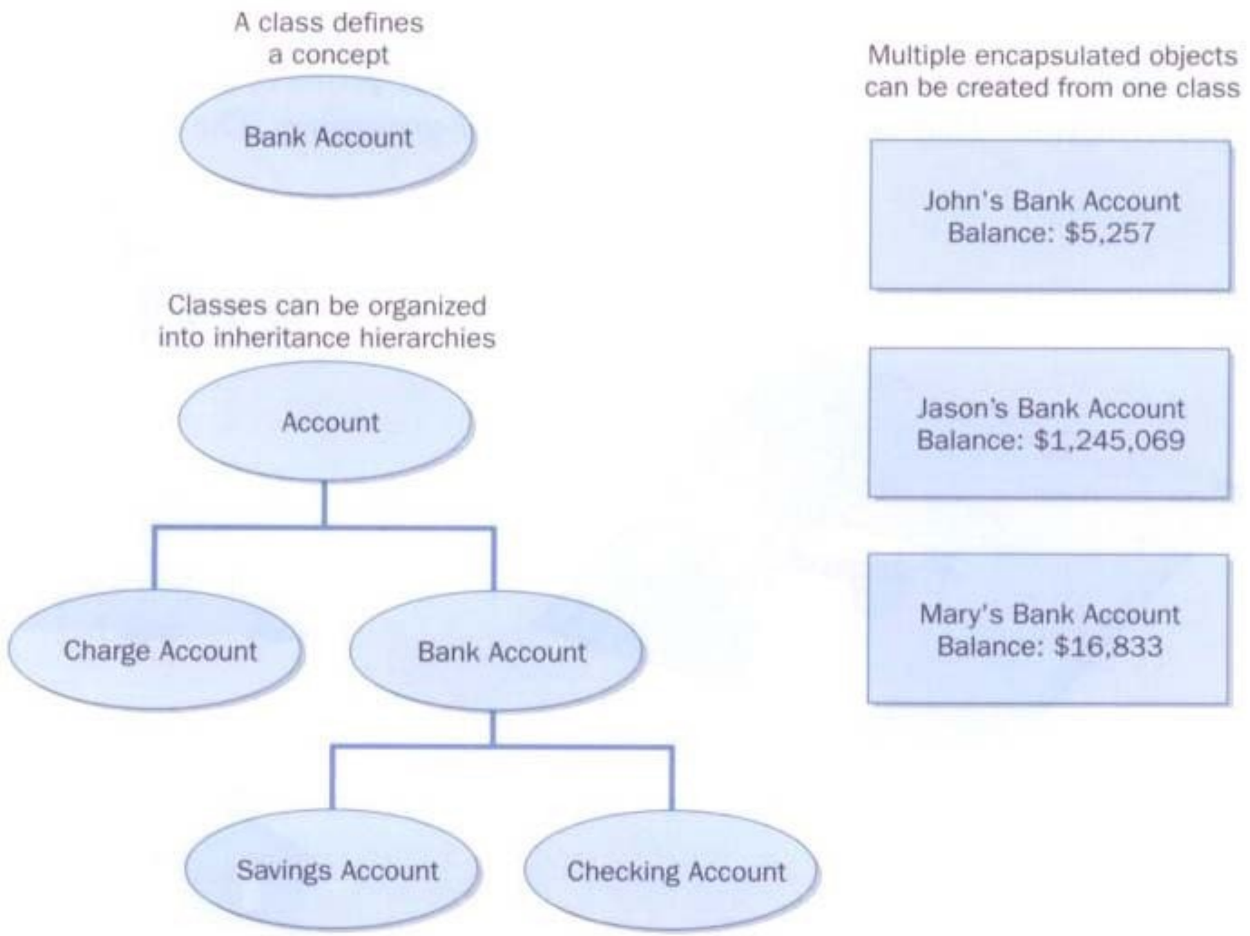
Multiple objects can be created from the same class

Classes and Objects

A class is like a blueprint from which you can create many of the "same" house with different characteristics



Classes and Objects Example



Classes and Objects

Fundamental things with Classes and Objects you need to know by now:

- Encapsulation: Visibility Modifier, Accessors and Mutators
- Constructors
- Attributes and Methods
- Static Class Members

Classes and Objects

Demo

Inheritance

Inheritance allows a software developer to derive a new class from an existing one

The existing class is called the *parent class,* or *superclass*, or *base class*

The derived class is called the *child class* or *subclass*

As the name implies, the child inherits characteristics of the parent

That is, the child class inherits the methods and data defined by the parent class

Inheritance

Things related to inheritance you need to know:

- Inheritance is the process of deriving a new class from an existing class
- Inheritance allows software reuse of existing software
- Inheritance creates an IS-A relationship between parent and child classes
- Protected visibility provides encapsulation and allows inheritance
- A parents constructor can and should be invoked by the child
- A child class can override a parents methods by redefining them
- A child of one class can be the parent of another
- All Java classes are derived from the `Object` class
- An Abstract class cannot be instantiated, it represents a concept
- Abstract classes must be declared in the child before it can be instantiated
- Private members of a class can not be seen directly in the child class
- The `final` modifier can be used to restrict inheritance

Inheritance

Demo

Polymorphism

- The term polymorphism literally means “having many forms”
- A polymorphic reference is a variable that can refer to different types of objects at different points in time
- The method invoked through a polymorphic reference can change from one invocation to the next
- All object references in Java are potentially polymorphic
- It is the type of the object being referenced, not the reference type, that determines which method is invoked
- Interfaces allow true encapsulation and hide implementation details
- We can have Polymorphism via Inheritance and Interface

Polymorphism

Demo